1. Manual Forward Pass and Backpropagation in Generator & Discriminator

```
import torch
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
import numpy as np
import matplotlib.pyplot as plt
import time
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cuda
```

```
# Hyperparameters
nz = 100           # Size of z latent vector
img_size = 28      # MNIST image size
img_flat_size = img_size * img_size
channels = 1       # MNIST is grayscale
```

```
lr_g = 0.0002      # Learning rate for Generator
lr_d = 0.0002      # Learning rate for Discriminator
```

```
batch_size = 128
num_epochs = 200
```

```
# For plotting generated images
fixed_noise = torch.randn(64, nz, device=device)
```

```
# Using MNIST dataset
# Normalizing to [-1, 1] as Tanh will be used in Generator output
transform_mnist = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)) # (mean, std)
])
```

```
print("Loading MNIST dataset...")
train_dataset = torchvision.datasets.MNIST(root='./data_mnist_gan', train=True,
                                           download=True, transform=transform_mnist)
dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
print("MNIST dataset loaded.")
```

```
Loading MNIST dataset...
100%|██████████| 9.91M/9.91M [00:00<00:00, 17.8MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 489kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.52MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 1.99MB/s]MNIST dataset loaded.
```

```
# Activation Functions
def relu(x):
    return torch.max(torch.tensor(0.0, device=x.device), x)

def leaky_relu(x, negative_slope=0.2):
    return torch.where(x > 0, x, x * negative_slope)

def tanh(x):
    return torch.tanh(x)

def sigmoid(x):
    return torch.sigmoid(x)
```

```
# Initializing weights
def weights_init_manual(tensor_shape):
    return torch.randn(tensor_shape, device=device) * 0.02
```

```
# Generator
H1_g_size = 256
H2_g_size = 512
```

```
W_g1 = weights_init_manual((nz, H1_g_size)).requires_grad_(True)
b_g1 = torch.zeros(H1_g_size, device=device).requires_grad_(True)
W_g2 = weights_init_manual((H1_g_size, H2_g_size)).requires_grad_(True)
b_g2 = torch.zeros(H2_g_size, device=device).requires_grad_(True)
W_g3 = weights_init_manual((H2_g_size, img_flat_size)).requires_grad_(True)
b_g3 = torch.zeros(img_flat_size, device=device).requires_grad_(True)
generator_params = [W_g1, b_g1, W_g2, b_g2, W_g3, b_g3]
```

```
# Discriminator
H1_d_size = 512
H2_d_size = 256
```

```
W_d1 = weights_init_manual((img_flat_size, H1_d_size)).requires_grad_(True)
b_d1 = torch.zeros(H1_d_size, device=device).requires_grad_(True)
W_d2 = weights_init_manual((H1_d_size, H2_d_size)).requires_grad_(True)
b_d2 = torch.zeros(H2_d_size, device=device).requires_grad_(True)
W_d3 = weights_init_manual((H2_d_size, 1)).requires_grad_(True)
b_d3 = torch.zeros(1, device=device).requires_grad_(True)
discriminator_params = [W_d1, b_d1, W_d2, b_d2, W_d3, b_d3]
```

```
print("Generator and Discriminator weights initialized manually.")
```

```
Generator and Discriminator weights initialized manually.
```

```
# Forward Pass Functions
# Using leakyrelu for discriminator and ReLU + Tanh for generator.
def generator_forward(z_input):
    h1 = relu(z_input @ W_g1 + b_g1)
    h2 = relu(h1 @ W_g2 + b_g2)
    img_output = tanh(h2 @ W_g3 + b_g3) # Tanh output
    return img_output.view(-1, channels, img_size, img_size) # Reshape to image
```

```
def discriminator_forward(img_input):
    img_flat = img_input.view(img_input.size(0), -1) # Flatten image
    h1 = leaky_relu(img_flat @ W_d1 + b_d1)
    h2 = leaky_relu(h1 @ W_d2 + b_d2)
    logit_output = h2 @ W_d3 + b_d3
    prob_output = sigmoid(logit_output)
    return prob_output # Output probability/
```

```
# Using Binary Cross Entropy Loss
epsilon = 1e-12 # To prevent log(0)
def bce_loss_manual(y_pred, y_true):
    y_pred_clipped = torch.clamp(y_pred, epsilon, 1.0 - epsilon)
    loss = -(y_true * torch.log(y_pred_clipped) + (1 - y_true) * torch.log(1 - y_pred_clipped))
    return torch.mean(loss)
```

```
# Writing custom training loops to update weights using SGD
# (Manual SGD implementation)
def sgd_step(params, lr):
    with torch.no_grad(): # We are manually updating, so no_grad for this part
        for p in params:
            if p.grad is not None:
                p.data.sub_(lr * p.grad.data) # p.data = p.data - lr * p.grad.data
                p.grad.zero_() # Zero out gradients after update
```

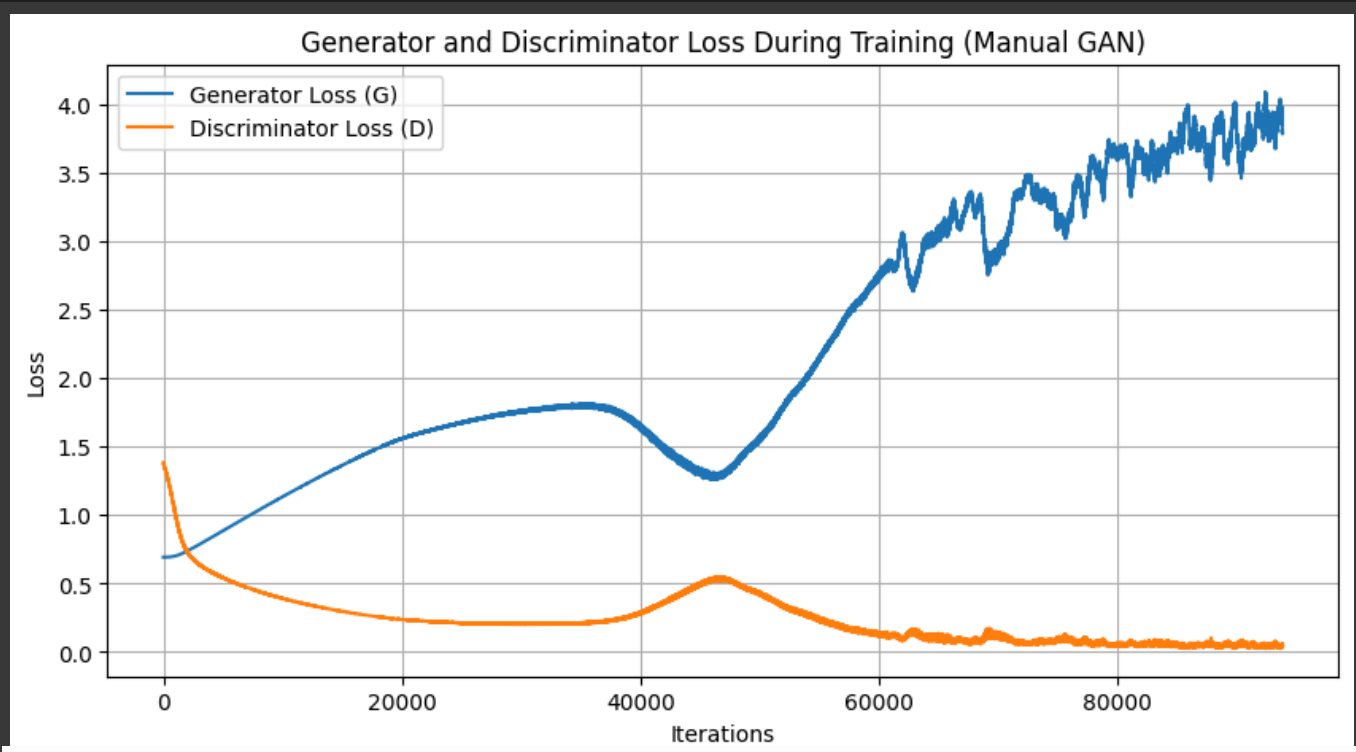
```
#Training Loop
G_losses = []
D_losses = []
img_list = [] # To store generated images at intervals
```

```
print("\nStarting GAN Training Loop (Manual)...")
for epoch in range(num_epochs):
    epoch_start_time = time.time()
    for i, (real_images, _) in enumerate(dataloader):
        real_images = real_images.to(device)
        current_batch_size = real_images.size(0)

        # Labels for loss calculation
        real_labels = torch.ones(current_batch_size, 1, device=device)
        fake_labels = torch.zeros(current_batch_size, 1, device=device)

        # Train Discriminator

        # On real images
        d_output_real = discriminator_forward(real_images)
        loss_d_real = bce_loss_manual(d_output_real, real_labels)

        # On fake images
        noise = torch.randn(current_batch_size, nz, device=device)
        fake_images = generator_forward(noise)
        # Detach fake_images so G's gradients are not calculated during D update
        d_output_fake = discriminator_forward(fake_images.detach())
        loss_d_fake = bce_loss_manual(d_output_fake, fake_labels)

        loss_d = loss_d_real + loss_d_fake

        # Manual backprop for D
        # Zero previous gradients for D params
        for p in discriminator_params:
            if p.grad is not None:
                p.grad.zero_()
        loss_d.backward()
        sgd_step(discriminator_params, lr_d)

        # Train Generator
        d_output_on_fake_for_g = discriminator_forward(fake_images) # fake_images here still has G's graph
        loss_g = bce_loss_manual(d_output_on_fake_for_g, real_labels) # Using real_labels for G's loss

        # Manual backprop for G
        # Zero previous gradients for G params
        for p in generator_params:
            if p.grad is not None:
                p.grad.zero_()
        loss_g.backward()
        sgd_step(generator_params, lr_g)

        # Save losses for plotting
        G_losses.append(loss_g.item())
        D_losses.append(loss_d.item())

        if (i + 1) % 100 == 0:
            print(f"Epoch [{epoch+1}/{num_epochs}], Batch [{i+1}/{len(dataloader)}], "
                  f"Loss D: {loss_d.item():.4f}, Loss G: {loss_g.item():.4f}")

    epoch_end_time = time.time()
    print(f"Epoch {epoch+1} finished in {epoch_end_time - epoch_start_time:.2f}s. "
          f"Avg D Loss: {np.mean(D_losses[-len(dataloader):]):.4f}, Avg G Loss: {np.mean(G_losses[-len(dataloader):]):.4f}")

    # Sample images at various epochs
    if epoch == 0 or (epoch + 1) % 25 == 0 or epoch == num_epochs - 1: # Adjusted sampling epochs
        with torch.no_grad():
```

```
        generated_samples = generator_forward(fixed_noise).detach().cpu()
        img_list.append(generated_samples)
        print(f"Generated samples at epoch {epoch+1} stored.")

print("Training complete.")
```

```
Epoch 139 finished in 14.43s. Avg D Loss: 0.1016, Avg G Loss: 3.0566
Epoch [140/200], Batch [100/469], Loss D: 0.0916, Loss G: 3.0581
Epoch [140/200], Batch [200/469], Loss D: 0.0887, Loss G: 3.1250
Epoch [140/200], Batch [300/469], Loss D: 0.1026, Loss G: 3.1252
Epoch [140/200], Batch [400/469], Loss D: 0.0824, Loss G: 3.0847
Epoch 140 finished in 13.73s. Avg D Loss: 0.0974, Avg G Loss: 3.0999
Epoch [141/200], Batch [100/469], Loss D: 0.0951, Loss G: 3.1340
Epoch [141/200], Batch [200/469], Loss D: 0.0812, Loss G: 3.1453
Epoch [141/200], Batch [300/469], Loss D: 0.0890, Loss G: 3.1782
Epoch [141/200], Batch [400/469], Loss D: 0.0860, Loss G: 3.1914
Epoch 141 finished in 13.59s. Avg D Loss: 0.0925, Avg G Loss: 3.1432
Epoch [142/200], Batch [100/469], Loss D: 0.0867, Loss G: 3.2847
Epoch [142/200], Batch [200/469], Loss D: 0.0718, Loss G: 3.2522
Epoch [142/200], Batch [300/469], Loss D: 0.0759, Loss G: 3.1910
Epoch [142/200], Batch [400/469], Loss D: 0.0915, Loss G: 3.1199
Epoch 142 finished in 13.67s. Avg D Loss: 0.0818, Avg G Loss: 3.2163
Epoch [143/200], Batch [100/469], Loss D: 0.0849, Loss G: 3.1134
Epoch [143/200], Batch [200/469], Loss D: 0.0876, Loss G: 3.0980
Epoch [143/200], Batch [300/469], Loss D: 0.0798, Loss G: 3.1336
Epoch [143/200], Batch [400/469], Loss D: 0.0800, Loss G: 3.1830
Epoch 143 finished in 13.83s. Avg D Loss: 0.0874, Avg G Loss: 3.1343
Epoch [144/200], Batch [100/469], Loss D: 0.0746, Loss G: 3.2285
Epoch [144/200], Batch [200/469], Loss D: 0.0780, Loss G: 3.2328
Epoch [144/200], Batch [300/469], Loss D: 0.0667, Loss G: 3.2946
Epoch [144/200], Batch [400/469], Loss D: 0.0724, Loss G: 3.3065
Epoch 144 finished in 13.77s. Avg D Loss: 0.0770, Avg G Loss: 3.2619
Epoch [145/200], Batch [100/469], Loss D: 0.0722, Loss G: 3.3252
Epoch [145/200], Batch [200/469], Loss D: 0.0721, Loss G: 3.3299
Epoch [145/200], Batch [300/469], Loss D: 0.0884, Loss G: 3.2375
Epoch [145/200], Batch [400/469], Loss D: 0.0807, Loss G: 3.2448
Epoch 145 finished in 14.42s. Avg D Loss: 0.0764, Avg G Loss: 3.2902
Epoch [146/200], Batch [100/469], Loss D: 0.0817, Loss G: 3.1887
Epoch [146/200], Batch [200/469], Loss D: 0.0762, Loss G: 3.2243
Epoch [146/200], Batch [300/469], Loss D: 0.0736, Loss G: 3.2857
Epoch [146/200], Batch [400/469], Loss D: 0.0780, Loss G: 3.3123
Epoch 146 finished in 13.68s. Avg D Loss: 0.0817, Avg G Loss: 3.2562
Epoch [147/200], Batch [100/469], Loss D: 0.0813, Loss G: 3.2502
Epoch [147/200], Batch [200/469], Loss D: 0.0888, Loss G: 3.1730
Epoch [147/200], Batch [300/469], Loss D: 0.1081, Loss G: 3.0419
Epoch [147/200], Batch [400/469], Loss D: 0.1292, Loss G: 2.9058
Epoch 147 finished in 14.50s. Avg D Loss: 0.0961, Avg G Loss: 3.1114
Epoch [148/200], Batch [100/469], Loss D: 0.1344, Loss G: 2.8469
Epoch [148/200], Batch [200/469], Loss D: 0.1337, Loss G: 2.8155
Epoch [148/200], Batch [300/469], Loss D: 0.1061, Loss G: 2.8446
Epoch [148/200], Batch [400/469], Loss D: 0.1168, Loss G: 2.8671
Epoch 148 finished in 13.92s. Avg D Loss: 0.1305, Avg G Loss: 2.8510
Epoch [149/200], Batch [100/469], Loss D: 0.1288, Loss G: 2.8493
Epoch [149/200], Batch [200/469], Loss D: 0.1073, Loss G: 2.9231
Epoch [149/200], Batch [300/469], Loss D: 0.1138, Loss G: 2.8804
Epoch [149/200], Batch [400/469], Loss D: 0.1100, Loss G: 2.8680
Epoch 149 finished in 15.29s. Avg D Loss: 0.1208, Avg G Loss: 2.8844
Epoch [150/200], Batch [100/469], Loss D: 0.1166, Loss G: 2.9117
Epoch [150/200], Batch [200/469], Loss D: 0.1211, Loss G: 2.9250
Epoch [150/200], Batch [300/469], Loss D: 0.1175, Loss G: 2.9662
Epoch [150/200], Batch [400/469], Loss D: 0.1067, Loss G: 2.9405
Epoch 150 finished in 13.79s. Avg D Loss: 0.1142, Avg G Loss: 2.9389
Generated samples at epoch 150 stored.
Epoch [151/200], Batch [100/469], Loss D: 0.0973, Loss G: 2.9767
Epoch [151/200], Batch [200/469], Loss D: 0.0972, Loss G: 3.0276
```

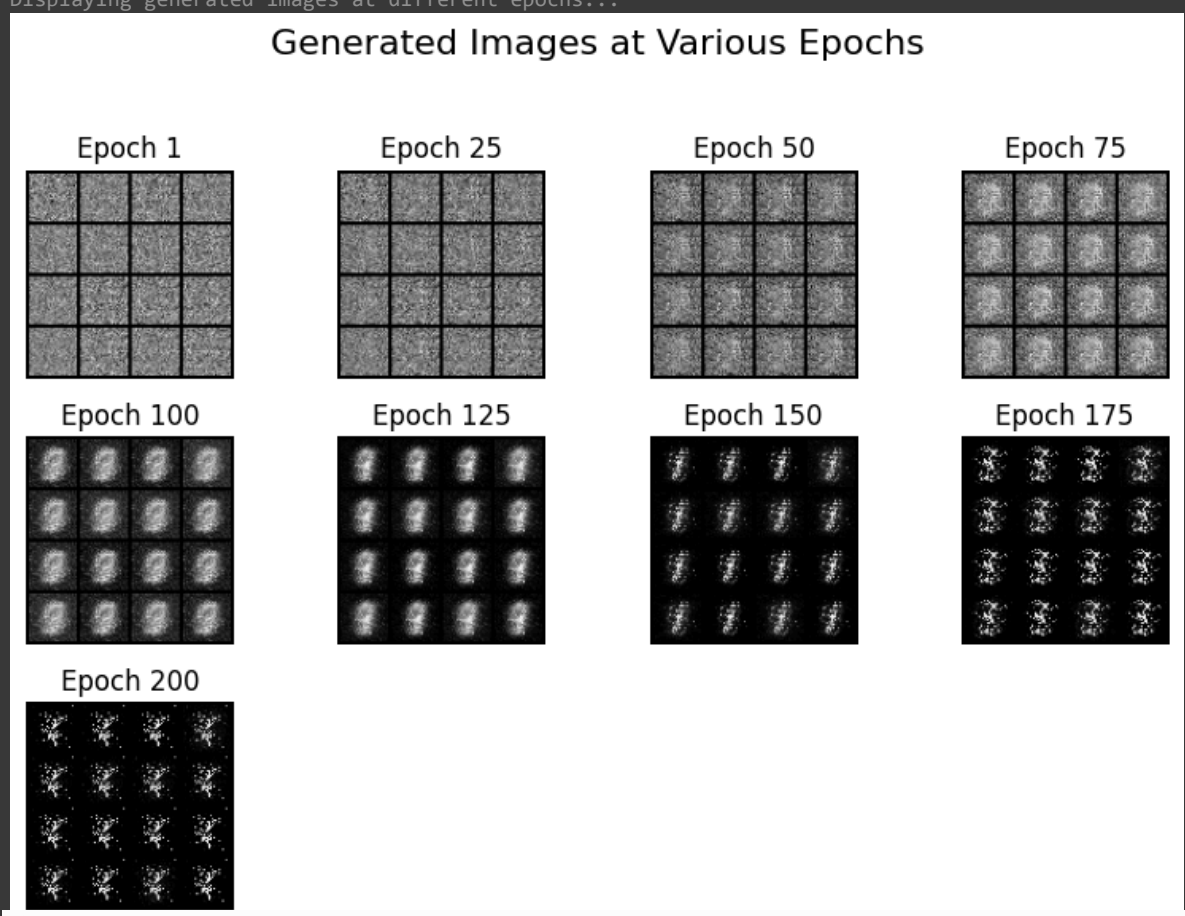
```
# Loss curves (generator vs discriminator)
plt.figure(figsize=(10, 5))
plt.title("Generator and Discriminator Loss During Training (Manual GAN)")
plt.plot(G_losses, label="Generator Loss (G)")
plt.plot(D_losses, label="Discriminator Loss (D)")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```




```
# Display Sampled Images
if 'img_list' in locals() and 'epochs_sampled_at' in locals() and len(img_list) > 0:
    print("\nDisplaying generated images from specified epochs (from img_list):")
    num_samples_to_show_from_batch = 16 # How many images from each sampled batch to show in the grid

    num_sampled_epochs = len(img_list)
    # Determine plot grid size
```

```
    cols_progression = min(4, num_sampled_epochs) # Show up to 4 epochs per row
    rows_progression = (num_sampled_epochs + cols_progression - 1) // cols_progression

    plt.figure(figsize=(cols_progression * 4, rows_progression * 4)) # Adjust size
    plt.suptitle("Progression of Generated MNIST Images at Specific Epochs", fontsize=16)

    for i, image_batch_tensor in enumerate(img_list):
        # Ensure we don't try to access an epoch number that doesn't exist
        epoch_label = f"Epoch {epochs_sampled_at[i]}" if i < len(epochs_sampled_at) else f"Sample {i+1}"

        plt.subplot(rows_progression, cols_progression, i + 1)
        # Create a grid from the batch of images
        grid = vutils.make_grid(image_batch_tensor[:num_samples_to_show_from_batch],
                                nrow=4, padding=2, normalize=True)
        plt.imshow(np.transpose(grid.cpu(), (1, 2, 0)))
        plt.title(epoch_label)
        plt.axis('off')

    plt.tight_layout(rect=[0, 0, 1, 0.95])
    plt.show()
else:
    print("Required 'img_list' or 'epochs_sampled_at' not found or empty. Skipping progression plot.")
```

```
Displaying generated images at different epochs...
```



Start coding or generate with AI.

2. Generate Augmented Data

```
import torch.nn as nn
import torch.optim as optim
import torchvision.datasets as dset # For MNIST
import torchvision.transforms as transforms
import torchvision.utils as vutils # For making image grids
import numpy as np
import matplotlib.pyplot as plt
import os
import time
```

```
# Hyperparameters
image_size = 28    # MNIST images are 28x28
nc = 1             # Number of channels
nz = 100           # Size of z latent vector
ngf = 32           # Size of feature maps in generator
ndf = 32
lr = 0.0002        # Learning rate for optimizers
beta1 = 0.5        # Beta1 hyperparameter
batch_size_gan = 128 # Batch size for GAN training
num_epochs_gan = 50
```

```
# Data Preparation: Loading MNIST and filtering for a selected class
selected_digit = 5 # Let's choose digit '5' as our target class to augment
print(f"Selected digit for augmentation: {selected_digit}")
```

```
Selected digit for augmentation: 5
```

```
# Transformation: ToTensor and Normalize to [-1, 1]
transform_gan = transforms.Compose([
    transforms.Resize(image_size),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

```
full_train_dataset = dset.MNIST(root='./data_mnist_gan_aug', train=True,
                                download=True, transform=transform_gan)
```

```
# Filter dataset for the selected digit
selected_digit_indices = [i for i, label in enumerate(full_train_dataset.targets) if label == selected_digit]
selected_digit_dataset = torch.utils.data.Subset(full_train_dataset, selected_digit_indices)

dataloader_selected_digit = torch.utils.data.DataLoader(selected_digit_dataset,
                                                        batch_size=batch_size_gan,
                                                        shuffle=True)
print(f"Created DataLoader for digit '{selected_digit}' with {len(selected_digit_dataset)} samples.")
```

```
Created DataLoader for digit '5' with 5421 samples.
```

```
# Plotting some images of the selected digit
if len(selected_digit_dataset) > 0:
    real_batch_selected_digit = next(iter(dataloader_selected_digit))
    plt.figure(figsize=(8,4))
    plt.axis("off")
    plt.title(f"Training Images for Digit '{selected_digit}'")
    plt.imshow(np.transpose(vutils.make_grid(real_batch_selected_digit[0].to(device)[:32],
                                             padding=2, normalize=True).cpu(),(1,2,0)))
    plt.show()
```

```
#GAN Model Definition

# Custom weights initialization
def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)
```

```
# Generator
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        self.main = nn.Sequential(
            nn.ConvTranspose2d(nz, ngf * 4, 7, 1, 0, bias=False),
            nn.BatchNorm2d(ngf * 4),
            nn.ReLU(True),

            nn.ConvTranspose2d(ngf * 4, ngf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 2),
            nn.ReLU(True),

            nn.ConvTranspose2d(ngf * 2, nc, 4, 2, 1, bias=False),
            nn.Tanh()

        )
    def forward(self, input_noise):
        return self.main(input_noise)
```

```
# Discriminator
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            nn.Conv2d(nc, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 2),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf * 2, 1, 7, 1, 0, bias=False),
            nn.Sigmoid()
        )
    def forward(self, input_image):
        return self.main(input_image)
```

```
# Create Generator and Discriminator
netG = Generator().to(device)
netD = Discriminator().to(device)
netG.apply(weights_init)
netD.apply(weights_init)
print("\nGenerator and Discriminator models created and weights initialized.")
```

```
Generator and Discriminator models created and weights initialized.
```

```
#Training Setup
criterion = nn.BCELoss() # Binary Cross Entropy loss
fixed_noise_gan = torch.randn(64, nz, 1, 1, device=device) # For visualizing G's progress
```

```
real_label_val = 1.
fake_label_val = 0.
```

```
optimizerD = optim.Adam(netD.parameters(), lr=lr, betas=(beta1, 0.999))
optimizerG = optim.Adam(netG.parameters(), lr=lr, betas=(beta1, 0.999))
print("Loss function and optimizers set up.")
```

```
Loss function and optimizers set up.
```

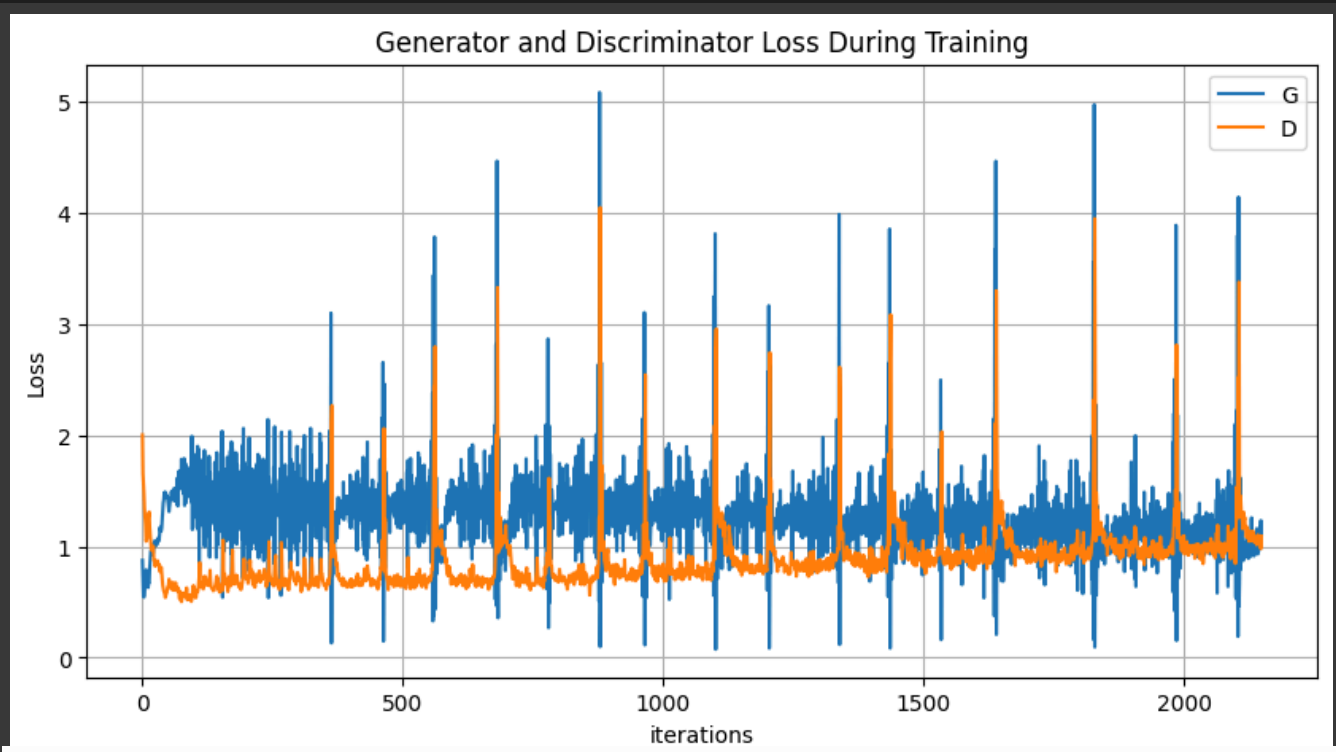
```
# --- 4. GAN Training Loop ---
G_losses = []
D_losses = []
img_list_gan = [] # To store generated images from fixed_noise
iters = 0
```

```
print(f"\nStarting GAN training for {num_epochs_gan} epochs on digit '{selected_digit}'...")
for epoch in range(num_epochs_gan):
    epoch_start_time = time.time()
    for i, data in enumerate(dataloader_selected_digit, 0):
        real_images_batch = data[0].to(device)
        b_size = real_images_batch.size(0)

        #Train Discriminator
        netD.zero_grad()
        # Real batch
        label_real = torch.full((b_size,), real_label_val, dtype=torch.float, device=device)
        output_real = netD(real_images_batch).view(-1)
        errD_real = criterion(output_real, label_real)
        errD_real.backward()
        D_x = output_real.mean().item()

        # Fake batch
        noise = torch.randn(b_size, nz, 1, 1, device=device)
        fake_images = netG(noise)
        label_fake = torch.full((b_size,), fake_label_val, dtype=torch.float, device=device)
        output_fake = netD(fake_images.detach()).view(-1)
        errD_fake = criterion(output_fake, label_fake)
        errD_fake.backward()
        D_G_z1 = output_fake.mean().item()
        errD = errD_real + errD_fake
        optimizerD.step()

        #Train Generator
        netG.zero_grad()
        label_real_for_G = torch.full((b_size,), real_label_val, dtype=torch.float, device=device) # G wants D to think fakes are real
        output_fake_for_G = netD(fake_images).view(-1) # Use the same fake_images, but don't detach
        errG = criterion(output_fake_for_G, label_real_for_G)
        errG.backward()
        D_G_z2 = output_fake_for_G.mean().item()
        optimizerG.step()

        if i % 50 == 0 and i > 0: # Printing less frequently for cleaner output
            print(f'Epoch [{epoch+1}/{num_epochs_gan}][{i}/{len(dataloader_selected_digit)}] '
                  f'Loss_D: {errD.item():.4f} Loss_G: {errG.item():.4f} '
                  f'D(x): {D_x:.4f} D(G(z)): {D_G_z1:.4f} / {D_G_z2:.4f}')

        G_losses.append(errG.item())
        D_losses.append(errD.item())
        iters += 1

    # Save generated images from fixed_noise at the end of some epochs
    if (epoch + 1) % 10 == 0 or epoch == num_epochs_gan - 1:
        with torch.no_grad():
            fake_viz = netG(fixed_noise_gan).detach().cpu()
        img_list_gan.append(vutils.make_grid(fake_viz, padding=2, normalize=True))

    epoch_end_time = time.time()
    print(f"Epoch {epoch+1} finished in {epoch_end_time - epoch_start_time:.2f}s. Avg D_Loss: {np.mean(D_losses[-len(dataloader_selected_digit):]):.4f}, Avg G_Loss: {np.mean(G_los

print("GAN Training complete.")
```

```
Starting GAN training for 50 epochs on digit '5'...
Epoch 1 finished in 2.91s. Avg D_Loss: 1.0091, Avg G_Loss: 0.9515
Epoch 2 finished in 1.74s. Avg D_Loss: 0.6027, Avg G_Loss: 1.5085
Epoch 3 finished in 1.75s. Avg D_Loss: 0.6227, Avg G_Loss: 1.4933
Epoch 4 finished in 1.72s. Avg D_Loss: 0.6755, Avg G_Loss: 1.4184
Epoch 5 finished in 1.80s. Avg D_Loss: 0.7316, Avg G_Loss: 1.3858
Epoch 6 finished in 2.16s. Avg D_Loss: 0.7341, Avg G_Loss: 1.3646
Epoch 7 finished in 1.76s. Avg D_Loss: 0.7094, Avg G_Loss: 1.3751
Epoch 8 finished in 1.78s. Avg D_Loss: 0.7015, Avg G_Loss: 1.3990
Epoch 9 finished in 1.77s. Avg D_Loss: 0.8309, Avg G_Loss: 1.3165
Epoch 10 finished in 1.77s. Avg D_Loss: 0.6830, Avg G_Loss: 1.3627
Epoch 11 finished in 1.75s. Avg D_Loss: 0.7903, Avg G_Loss: 1.3673
Epoch 12 finished in 1.96s. Avg D_Loss: 0.6893, Avg G_Loss: 1.3750
Epoch 13 finished in 2.06s. Avg D_Loss: 0.7103, Avg G_Loss: 1.3734
Epoch 14 finished in 1.74s. Avg D_Loss: 0.9962, Avg G_Loss: 1.3009
Epoch 15 finished in 1.76s. Avg D_Loss: 0.7002, Avg G_Loss: 1.4019
Epoch 16 finished in 1.71s. Avg D_Loss: 0.8673, Avg G_Loss: 1.4315
Epoch 17 finished in 1.73s. Avg D_Loss: 0.8707, Avg G_Loss: 1.2421
Epoch 18 finished in 1.74s. Avg D_Loss: 0.7009, Avg G_Loss: 1.4004
Epoch 19 finished in 2.22s. Avg D_Loss: 0.8018, Avg G_Loss: 1.3803
Epoch 20 finished in 1.85s. Avg D_Loss: 0.7220, Avg G_Loss: 1.4089
Epoch 21 finished in 1.69s. Avg D_Loss: 0.9742, Avg G_Loss: 1.4160
Epoch 22 finished in 1.77s. Avg D_Loss: 0.7569, Avg G_Loss: 1.3331
Epoch 23 finished in 1.70s. Avg D_Loss: 0.9121, Avg G_Loss: 1.3386
Epoch 24 finished in 1.72s. Avg D_Loss: 0.7797, Avg G_Loss: 1.3459
Epoch 25 finished in 1.92s. Avg D_Loss: 0.7861, Avg G_Loss: 1.3346
Epoch 26 finished in 2.16s. Avg D_Loss: 1.0795, Avg G_Loss: 1.2905
Epoch 27 finished in 1.79s. Avg D_Loss: 0.8995, Avg G_Loss: 1.2470
Epoch 28 finished in 1.77s. Avg D_Loss: 0.8359, Avg G_Loss: 1.3616
Epoch 29 finished in 1.79s. Avg D_Loss: 0.9520, Avg G_Loss: 1.2233
Epoch 30 finished in 1.73s. Avg D_Loss: 0.8304, Avg G_Loss: 1.2842
Epoch 31 finished in 1.73s. Avg D_Loss: 0.8449, Avg G_Loss: 1.2885
Epoch 32 finished in 2.00s. Avg D_Loss: 1.0111, Avg G_Loss: 1.2507
```

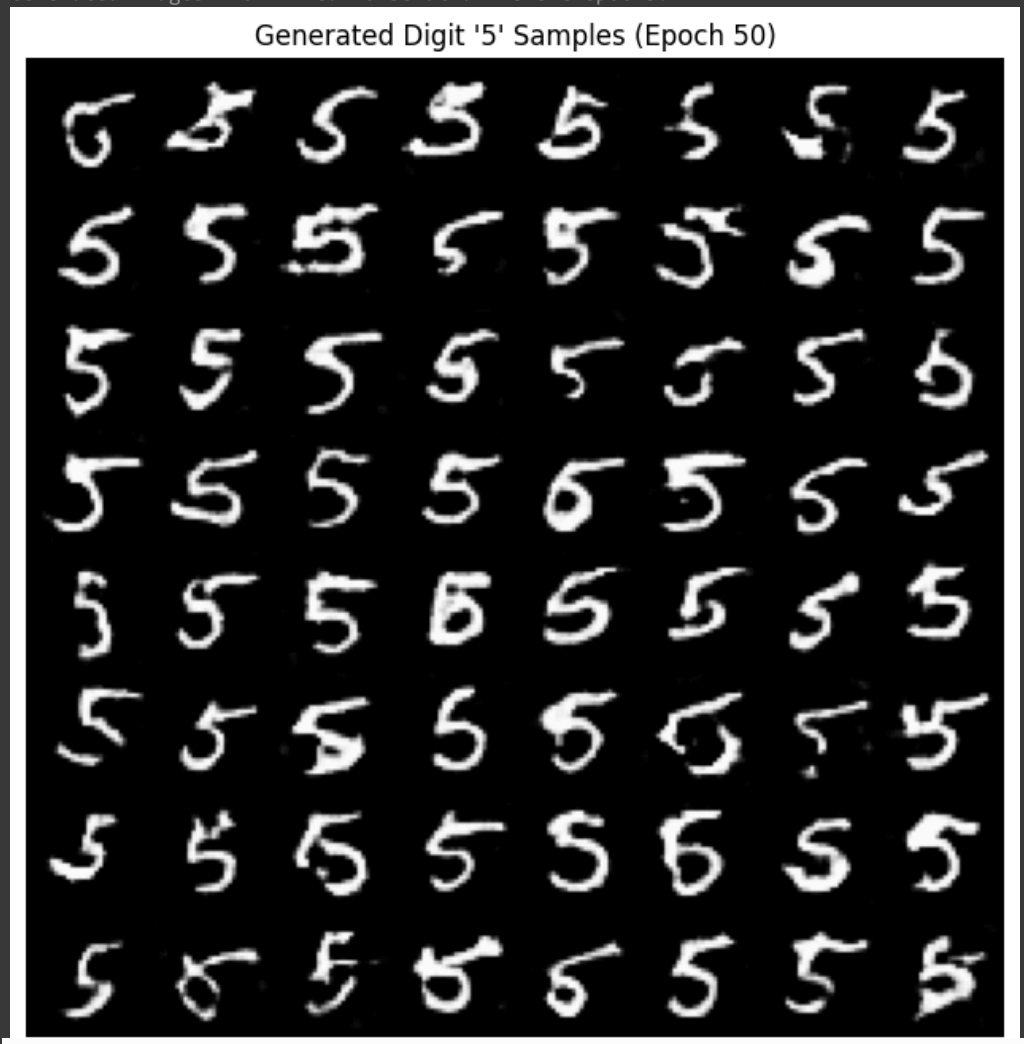
```
Epoch 33 finished in 2.01s. Avg D_Loss: 0.8540, Avg G_Loss: 1.2732
Epoch 34 finished in 1.73s. Avg D_Loss: 1.2028, Avg G_Loss: 1.2329
Epoch 35 finished in 1.72s. Avg D_Loss: 0.9477, Avg G_Loss: 1.1943
Epoch 36 finished in 2.30s. Avg D_Loss: 0.9532, Avg G_Loss: 1.2233
Epoch 37 finished in 1.75s. Avg D_Loss: 0.9134, Avg G_Loss: 1.2439
Epoch 38 finished in 1.87s. Avg D_Loss: 0.9061, Avg G_Loss: 1.2387
Epoch 39 finished in 2.09s. Avg D_Loss: 1.2404, Avg G_Loss: 1.1543
Epoch 40 finished in 1.76s. Avg D_Loss: 0.9229, Avg G_Loss: 1.1694
Epoch 41 finished in 1.77s. Avg D_Loss: 0.9349, Avg G_Loss: 1.2158
Epoch 42 finished in 1.74s. Avg D_Loss: 0.9432, Avg G_Loss: 1.2047
Epoch 43 finished in 1.72s. Avg D_Loss: 1.2132, Avg G_Loss: 1.2254
Epoch 44 finished in 1.75s. Avg D_Loss: 1.0042, Avg G_Loss: 1.1021
Epoch 45 finished in 2.08s. Avg D_Loss: 0.9785, Avg G_Loss: 1.1635
Epoch 46 finished in 1.92s. Avg D_Loss: 0.9640, Avg G_Loss: 1.1537
Epoch 47 finished in 1.76s. Avg D_Loss: 1.1314, Avg G_Loss: 1.1738
Epoch 48 finished in 1.72s. Avg D_Loss: 0.9766, Avg G_Loss: 1.1392
Epoch 49 finished in 1.73s. Avg D_Loss: 1.1372, Avg G_Loss: 1.2714
Epoch 50 finished in 1.74s. Avg D_Loss: 1.1387, Avg G_Loss: 1.0166
GAN Training complete.
```

```
# Plot G & D losses
plt.figure(figsize=(10,5))
plt.title("Generator and Discriminator Loss During Training")
plt.plot(G_losses,label="G")
plt.plot(D_losses,label="D")
plt.xlabel("iterations")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```




```
# Visualize some generated images from fixed noise over epochs
if img_list_gan:
    print("\nGenerated images from fixed noise at different epochs:")
    fig = plt.figure(figsize=(8,8))
    plt.axis("off")
    # Display the last saved grid (latest epoch)
    plt.imshow(np.transpose(img_list_gan[-1],(1,2,0)))
    plt.title(f"Generated Digit '{selected_digit}' Samples (Epoch {num_epochs_gan})")
    plt.show()
```

```
Generated images from fixed noise at different epochs:
```



```
#  Generate 1000+ new synthetic images
print("\nTask 1: Generating 1000+ synthetic images...")
num_synthetic_images = 1100
netG.eval()
generated_images_list = []
with torch.no_grad():
    for _ in range(int(np.ceil(num_synthetic_images / batch_size_gan))):
        noise = torch.randn(batch_size_gan, nz, 1, 1, device=device)
        synthetic_batch = netG(noise).cpu()
        generated_images_list.append(synthetic_batch)

generated_images_tensor = torch.cat(generated_images_list, dim=0)[:num_synthetic_images]
print(f"Generated {generated_images_tensor.shape[0]} synthetic images of shape {generated_images_tensor.shape[1:]}")
```

```
Task 1: Generating 1000+ synthetic images...
Generated 1100 synthetic images of shape torch.Size([1, 28, 28])
```

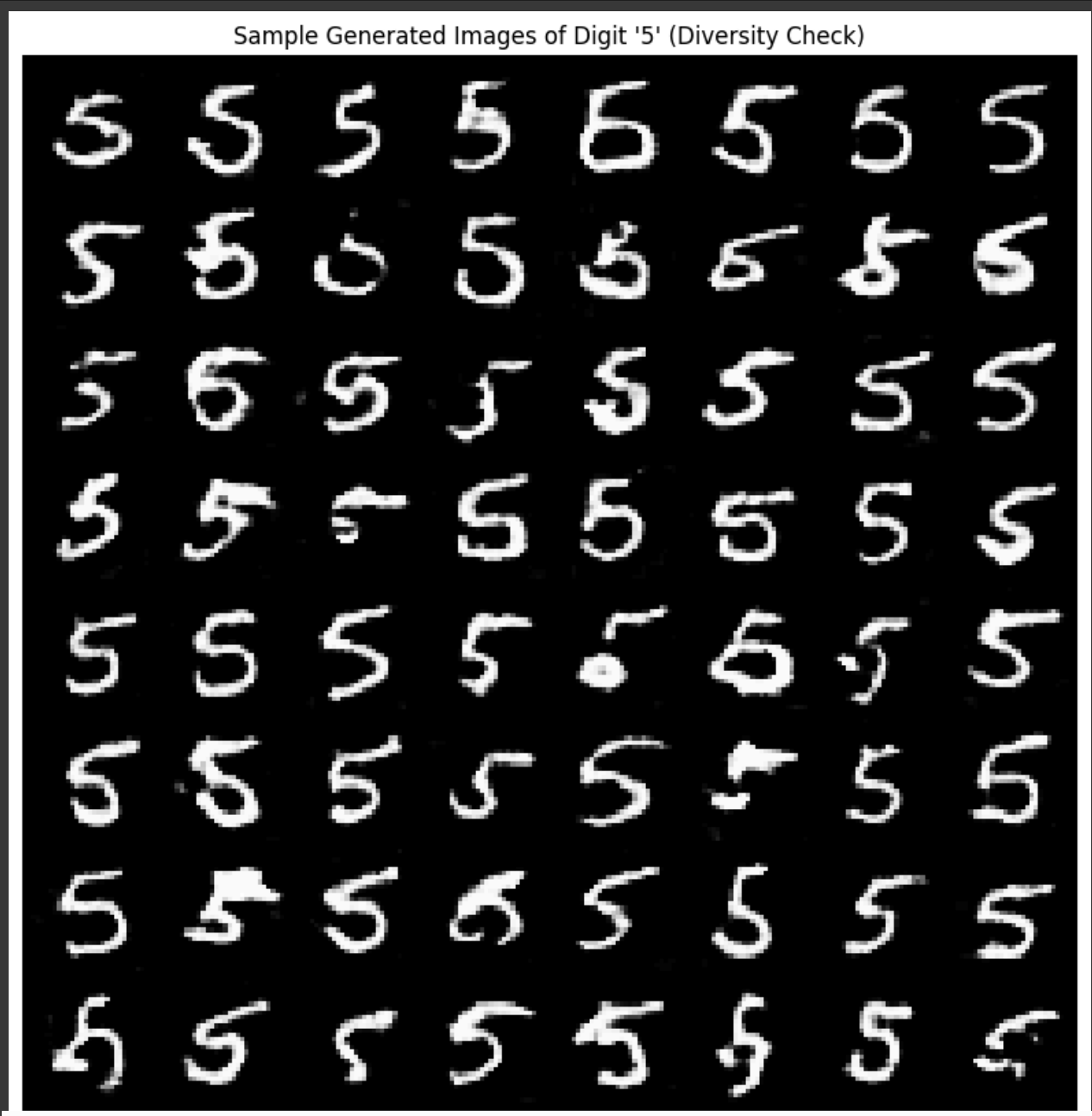
```
# Save these images and visually verify diversity
save_dir = f'./generated_digit_{selected_digit}'
os.makedirs(save_dir, exist_ok=True)
print(f"\nTask 2: Saving a subset of generated images to '{save_dir}' and visualizing...")
```

```
Task 2: Saving a subset of generated images to './generated_digit_5' and visualizing...
```

```
for i in range(min(100, num_synthetic_images)):
    vutils.save_image(generated_images_tensor[i], f'{save_dir}/synth_{selected_digit}_{i+1:04d}.png', normalize=True)
print(f"Saved {min(100, num_synthetic_images)} images.")
```

```
Saved 100 images.
```

```
plt.figure(figsize=(10,10))
plt.axis("off")
plt.title(f"Sample Generated Images of Digit '{selected_digit}' (Diversity Check)")
plt.imshow(np.transpose(vutils.make_grid(generated_images_tensor[:64], padding=2, normalize=True),(1,2,0)))
plt.show()
```




```
# Combining with the original dataset to create a new balanced dataset
print(f"\nCombining generated images with the original MNIST training set...")
generated_labels = torch.full((num_synthetic_images,), selected_digit, dtype=torch.long)
```

```
Combining generated images with the original MNIST training set...
```

```
temp_full_loader = torch.utils.data.DataLoader(full_train_dataset, batch_size=len(full_train_dataset), shuffle=False)
original_images_all, original_labels_all = next(iter(temp_full_loader))
```

```
augmented_images = torch.cat((original_images_all, generated_images_tensor), dim=0)
augmented_labels = torch.cat((original_labels_all, generated_labels), dim=0)
```

```
augmented_dataset = torch.utils.data.TensorDataset(augmented_images, augmented_labels)
print(f"Created augmented dataset with {len(augmented_dataset)} total samples.")
```

```
Created augmented dataset with 61100 total samples.
```

```
unique_labels, counts = np.unique(augmented_labels.numpy(), return_counts=True)
print("\nNew dataset class distribution:")
for label, count in zip(unique_labels, counts):
    print(f"  Digit {label}: {count} samples")
```

```
New dataset class distribution:
  Digit 0: 5923 samples
  Digit 1: 6742 samples
  Digit 2: 5958 samples
  Digit 3: 6131 samples
  Digit 4: 5842 samples
  Digit 5: 6521 samples
  Digit 6: 5918 samples
  Digit 7: 6265 samples
  Digit 8: 5851 samples
  Digit 9: 5949 samples
```

```
augmented_dataloader = torch.utils.data.DataLoader(augmented_dataset, batch_size=batch_size_gan, shuffle=True)
print(f"Created DataLoader for the augmented dataset.")
```

```
Created DataLoader for the augmented dataset.
```

3. Train a Classifier from Scratch on Original vs Augmented Dataset

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
#Hyperparameters
input_size_clf = 28 * 28
hidden_size_clf = 128
output_size_clf = 10
lr_clf = 0.01
epochs_clf = 10
batch_size_clf = 64
```

```
print("Loading MNIST test dataset for Q3 evaluation...")
transform_mnist_test_clf = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])
mnist_test_dataset = dset.MNIST(root='./data_mnist_test_clf', train=False,
                                download=True, transform=transform_mnist_test_clf)

original_train_loader = DataLoader(full_train_dataset, batch_size=batch_size_clf, shuffle=True)
augmented_train_loader = DataLoader(augmented_dataset, batch_size=batch_size_clf, shuffle=True)
mnist_test_loader = DataLoader(mnist_test_dataset, batch_size=batch_size_clf, shuffle=False)
```

```
Loading MNIST test dataset for Q3 evaluation...
100%|██████████| 9.91M/9.91M [00:00<00:00, 16.7MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 495kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.53MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 6.16MB/s]
```

```
#  Manually implement a 2-layer classifier
# 1 hidden layer + 1 output layer

def init_manual_classifier_weights(input_s, hidden_s, output_s):
    W1 = torch.randn(input_s, hidden_s, device=device) / np.sqrt(input_s)
    b1 = torch.zeros(hidden_s, device=device)
    W2 = torch.randn(hidden_s, output_s, device=device) / np.sqrt(hidden_s)
    b2 = torch.zeros(output_s, device=device)
    return W1.requires_grad_(True), b1.requires_grad_(True), \
           W2.requires_grad_(True), b2.requires_grad_(True)
```

```
def relu_manual(x):
    return torch.max(torch.tensor(0.0, device=x.device), x)
```

```
def classifier_forward_manual(x_batch_flat, W1, b1, W2, b2):
    # Layer 1
    h_pre_activation = torch.matmul(x_batch_flat, W1) + b1
    h_activated = relu_manual(h_pre_activation)
    # Layer 2 (Output logits)
    logits = torch.matmul(h_activated, W2) + b2
    return logits
```

```
def cross_entropy_loss_manual(logits, y_true_labels):
    batch_s = logits.shape[0]
    log_sum_exp = torch.log(torch.sum(torch.exp(logits), dim=1, keepdim=True) + 1e-9) # Add epsilon for stability
    log_softmax_val = logits - log_sum_exp
    nll = -log_softmax_val[range(batch_s), y_true_labels]
    return torch.mean(nll)
```

```
# Manual SGD update
def sgd_update_manual(params_list, learning_rate):
    with torch.no_grad():
        for p in params_list:
            if p.grad is not None:
                p.data -= learning_rate * p.grad.data # In-place subtraction
                p.grad.zero_() # Clear gradients after update
```

```
# Training Function for Manual Classifier
def train_manual_classifier(params, dataloader, num_epochs, learning_rate, loss_fn):
    W1, b1, W2, b2 = params
    losses_hist = []
    acc_hist = []
    print(f"Starting manual classifier training for {num_epochs} epochs...")
    for epoch in range(num_epochs):
        epoch_loss = 0.0
        epoch_correct = 0
        epoch_total = 0
        for images, labels in dataloader:
```

```
            images, labels = images.to(device), labels.to(device)
            images_flat = images.view(images.size(0), -1) # Flatten

            # Forward pass
            logits = classifier_forward_manual(images_flat, W1, b1, W2, b2)
            loss = loss_fn(logits, labels)

            # Backward pass
            # Zero previous gradients before calling loss.backward()
            for p in params:
                if p.grad is not None:
                    p.grad.zero_()
            loss.backward()

            # Update weights
            sgd_update_manual(params, learning_rate)

            epoch_loss += loss.item() * images.size(0)
            _, predicted = torch.max(logits.data, 1)
            epoch_total += labels.size(0)
            epoch_correct += (predicted == labels).sum().item()

        avg_epoch_loss = epoch_loss / epoch_total
        avg_epoch_acc = 100. * epoch_correct / epoch_total
        losses_hist.append(avg_epoch_loss)
        acc_hist.append(avg_epoch_acc)
        print(f"  Epoch [{epoch+1}/{num_epochs}], Loss: {avg_epoch_loss:.4f}, Accuracy: {avg_epoch_acc:.2f}%")
    print("Training complete.")
    return losses_hist, acc_hist
```

```
# Evaluation Function for Manual Classifier
def evaluate_manual_classifier(params, dataloader, loss_fn):
    W1, b1, W2, b2 = params
    all_preds = []
    all_true = []
    eval_loss = 0.0
    eval_correct = 0
    eval_total = 0
    with torch.no_grad(): # No gradients needed for evaluation
        for images, labels in dataloader:
            images, labels = images.to(device), labels.to(device)
            images_flat = images.view(images.size(0), -1)

            logits = classifier_forward_manual(images_flat, W1, b1, W2, b2)
            loss = loss_fn(logits, labels)
            eval_loss += loss.item() * images.size(0)

            _, predicted = torch.max(logits.data, 1)
            eval_total += labels.size(0)
            eval_correct += (predicted == labels).sum().item()

            all_preds.extend(predicted.cpu().numpy())
            all_true.extend(labels.cpu().numpy())

    avg_eval_loss = eval_loss / eval_total
    avg_eval_acc = 100. * eval_correct / eval_total
    print(f"  Evaluation - Loss: {avg_eval_loss:.4f}, Accuracy: {avg_eval_acc:.2f}%")
    return avg_eval_acc, all_true, all_preds
```

```
#  Train on original dataset and evaluate
print("\n--- Training Classifier on ORIGINAL MNIST Dataset ---")
W1_orig, b1_orig, W2_orig, b2_orig = init_manual_classifier_weights(input_size_clf, hidden_size_clf, output_size_clf)
params_orig = [W1_orig, b1_orig, W2_orig, b2_orig]
train_losses_orig, train_accs_orig = train_manual_classifier(params_orig, original_train_loader, epochs_clf, lr_clf, cross_entropy_loss_manual)
```

```
--- Training Classifier on ORIGINAL MNIST Dataset ---
Starting manual classifier training for 10 epochs...
  Epoch [1/10], Loss: 0.6493, Accuracy: 83.11%
  Epoch [2/10], Loss: 0.3480, Accuracy: 90.04%
  Epoch [3/10], Loss: 0.3059, Accuracy: 91.16%
  Epoch [4/10], Loss: 0.2804, Accuracy: 91.95%
  Epoch [5/10], Loss: 0.2602, Accuracy: 92.61%
  Epoch [6/10], Loss: 0.2419, Accuracy: 93.12%
  Epoch [7/10], Loss: 0.2262, Accuracy: 93.61%
  Epoch [8/10], Loss: 0.2119, Accuracy: 94.02%
  Epoch [9/10], Loss: 0.1988, Accuracy: 94.42%
  Epoch [10/10], Loss: 0.1872, Accuracy: 94.83%
Training complete.
```

```
print("\nEvaluating on Test Set (trained on ORIGINAL data):")
acc_orig, true_orig, pred_orig = evaluate_manual_classifier(params_orig, mnist_test_loader, cross_entropy_loss_manual)
```

```
Evaluating on Test Set (trained on ORIGINAL data):
  Evaluation - Loss: 0.1868, Accuracy: 94.57%
```

```
# Using accuracy, precision, or confusion matrix for evaluation.
print("\nClassification Report (trained on ORIGINAL data):")
print(classification_report(true_orig, pred_orig, digits=3))
```

```
Classification Report (trained on ORIGINAL data):
              precision    recall  f1-score   support

           0      0.957     0.984     0.970       980
           1      0.985     0.977     0.981      1135
           2      0.946     0.937     0.942      1032
           3      0.954     0.932     0.943      1010
           4      0.916     0.955     0.935       982
           5      0.951     0.918     0.934       892
           6      0.935     0.959     0.947       958
           7      0.942     0.945     0.943      1028
           8      0.914     0.944     0.928       974
           9      0.954     0.902     0.927      1009
```

```
    accuracy                           0.946     10000
   macro avg      0.945     0.945     0.945     10000
weighted avg      0.946     0.946     0.946     10000
```

```
# Train on augmented dataset and compare
print("\n--- Training Classifier on AUGMENTED MNIST Dataset ---")
# Re-initialize weights for a fair comparison
W1_aug, b1_aug, W2_aug, b2_aug = init_manual_classifier_weights(input_size_clf, hidden_size_clf, output_size_clf)
params_aug = [W1_aug, b1_aug, W2_aug, b2_aug]
train_losses_aug, train_accs_aug = train_manual_classifier(params_aug, augmented_train_loader, epochs_clf, lr_clf, cross_entropy_loss_manual)
```

```
--- Training Classifier on AUGMENTED MNIST Dataset ---
Starting manual classifier training for 10 epochs...
  Epoch [1/10], Loss: 0.6489, Accuracy: 82.83%
  Epoch [2/10], Loss: 0.3511, Accuracy: 89.97%
  Epoch [3/10], Loss: 0.3081, Accuracy: 91.10%
  Epoch [4/10], Loss: 0.2806, Accuracy: 91.89%
  Epoch [5/10], Loss: 0.2593, Accuracy: 92.61%
  Epoch [6/10], Loss: 0.2401, Accuracy: 93.17%
  Epoch [7/10], Loss: 0.2238, Accuracy: 93.63%
  Epoch [8/10], Loss: 0.2094, Accuracy: 94.07%
  Epoch [9/10], Loss: 0.1957, Accuracy: 94.43%
  Epoch [10/10], Loss: 0.1845, Accuracy: 94.75%
Training complete.
```

```
print("\nEvaluating on Test Set (trained on AUGMENTED data):")
acc_aug, true_aug, pred_aug = evaluate_manual_classifier(params_aug, mnist_test_loader, cross_entropy_loss_manual)
```

```
Evaluating on Test Set (trained on AUGMENTED data):
  Evaluation - Loss: 0.1786, Accuracy: 94.87%
```

```
print("\nClassification Report (trained on AUGMENTED data):")
print(classification_report(true_aug, pred_aug, digits=3))
```

```
Classification Report (trained on AUGMENTED data):
              precision    recall  f1-score   support

           0      0.973     0.978     0.975       980
           1      0.967     0.982     0.975      1135
           2      0.940     0.956     0.948      1032
           3      0.938     0.951     0.945      1010
           4      0.920     0.961     0.940       982
           5      0.946     0.916     0.931       892
           6      0.949     0.965     0.957       958
           7      0.955     0.936     0.945      1028
           8      0.938     0.929     0.933       974
           9      0.960     0.906     0.932      1009

    accuracy                          0.949     10000
   macro avg      0.949     0.948     0.948     10000
weighted avg      0.949     0.949     0.949     10000
```

```
# Comparison
print("\n--- Comparison of Test Accuracies ---")
print(f"Accuracy on Test Set (trained on ORIGINAL MNIST):   {acc_orig:.2f}%")
print(f"Accuracy on Test Set (trained on AUGMENTED MNIST): {acc_aug:.2f}%")
```

```
--- Comparison of Test Accuracies ---
Accuracy on Test Set (trained on ORIGINAL MNIST):   94.57%
Accuracy on Test Set (trained on AUGMENTED MNIST): 94.87%
```

```
if acc_aug > acc_orig:
    print("The model trained on the augmented dataset performed better on the test set.")
elif acc_orig > acc_aug:
    print("The model trained on the original dataset performed better on the test set.")
else:
    print("Both models performed similarly on the test set.")
```

```
The model trained on the augmented dataset performed better on the test set.
```

Start coding or generate with AI.